



Persistent

Angular Components

Persistent Interactive | Persistent University



Key Learning Points

- What are Angular Components
- How to implement a Component
- Properties of @Component
- Importance of selector
- Component Tree structure

Introduction to Components

- Components are the basic building blocks of an Angular application.
- Components are visible on the browser and define a specific part of the web page.
- Every Component has some properties, behavior and a view.

Examples

Login Page

- Form to take input of username and password.
- Method login() which will internally invoke an authentication service.
- Displays Error message if username and password incorrect.
- Redirects to next page on successful login.

Shopping Cart

- List of items selected by user.
- Remove from cart.
- Modify quantity.

Examples

LoginComponent

- **Model:** Username, password.
- **View:** Form with 2 inputs and 1 submit button.
- **Controllers:** Methods login(), cancel().

CartComponent

- **Model:** List of selected items.
- **View:** Ordered list or a table.
- **Controllers:** editItem(), removeItem().

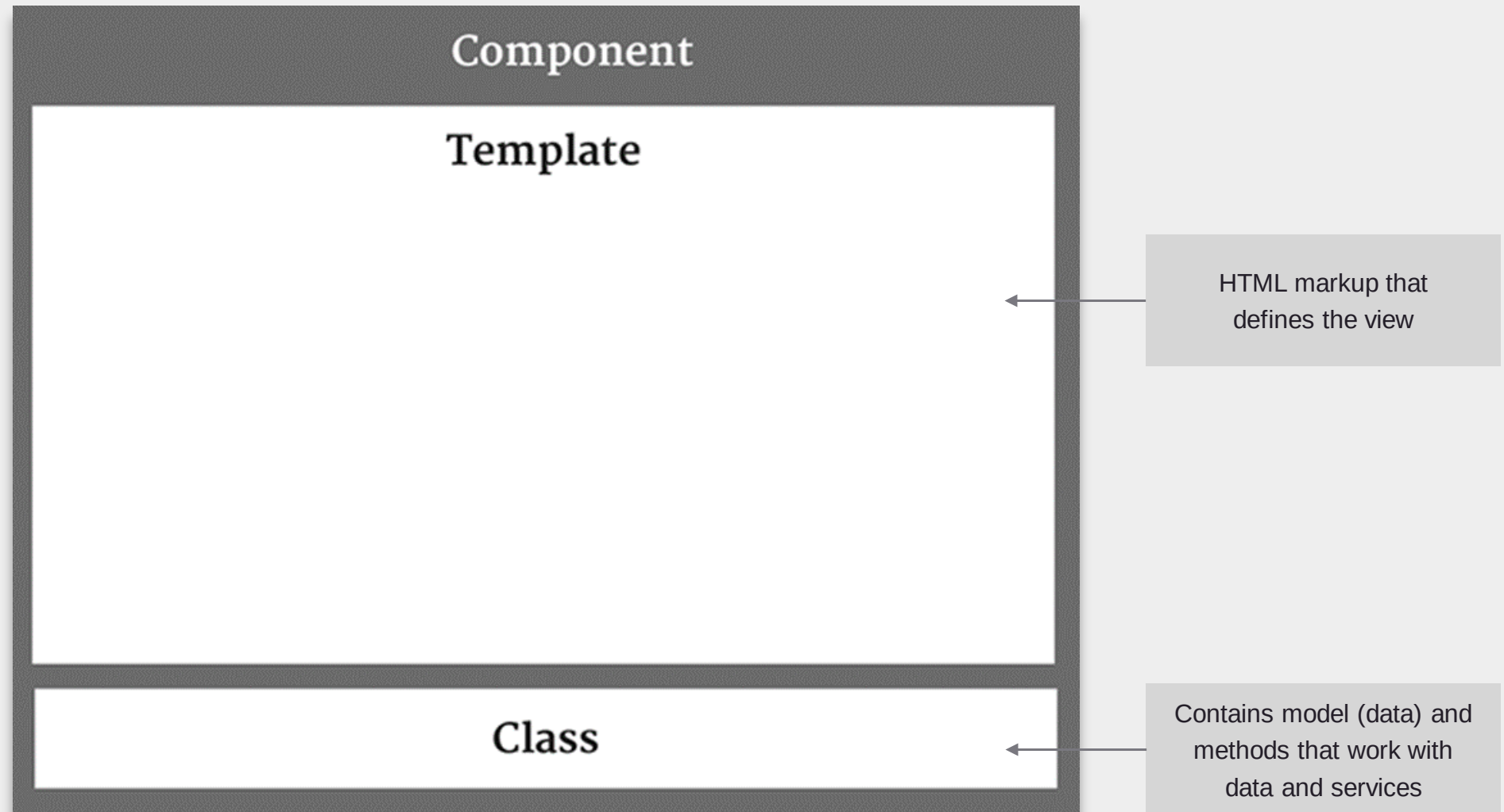
More Examples...

- Registration Form
- Menu Bar
- User Cart
- User Profile
- Page Header

How to Implement a Component?

- Component is a simple TypeScript class.
- Component is a class having properties, behavior and a view.
- Build Re-usable components.
- Component files are named with .component suffix.

Template and Class



Annotation of @Component

- Template can be inline or can point to an html page as well.
- @Component marks it as an Angular Component.

```
import { Component } from '@angular/core';  
@Component({  
  selector: 'my-app',  
  template: '<h1>Hello {{name}}</h1>',  
})  
export class AppComponent {  
  name = 'Angular';  
}
```

Properties of @Component Annotation

Selector:	HTML tag for the component
Template/templateUrl:	HTML markup for the View
Styles:	CSS Styles for the View
Providers:	Services injected in the Component

Component Selectors

- Every HTML element has a tag associated with it.
 - `<div></div>`, `<form></form>`
- Every component too has a selector.
 - Eg: LoginComponent → login,
 - Eg: RegistrationComponent → reg-form
- To add a component to page use the associated selector.
 - Eg. `<login></login>`
 - Eg: `<reg-form></reg-form>`
- The index.html page has the selector `<app-root></app-root>`

Interpolation

- Interpolation is used to display component properties in the views.
- Let's say, we have a component property as “name”.

```
export class AppComponent {  
    name:string = 'Angular';  
}
```

- We display the “name” using interpolation syntax of {{}}

```
@Component({  
    selector: 'my-app',  
    template: `<h1>Hello {{name}}</h1>`,  
})
```

Displaying Data

- Used in Element values

```
<p>Welcome to Angular {{name}}</p>
```

- Used in attribute values

```

```

- Template expressions are evaluated, converted into strings and then appended to neighboring element.
- Context of these expressions are in Component instance.

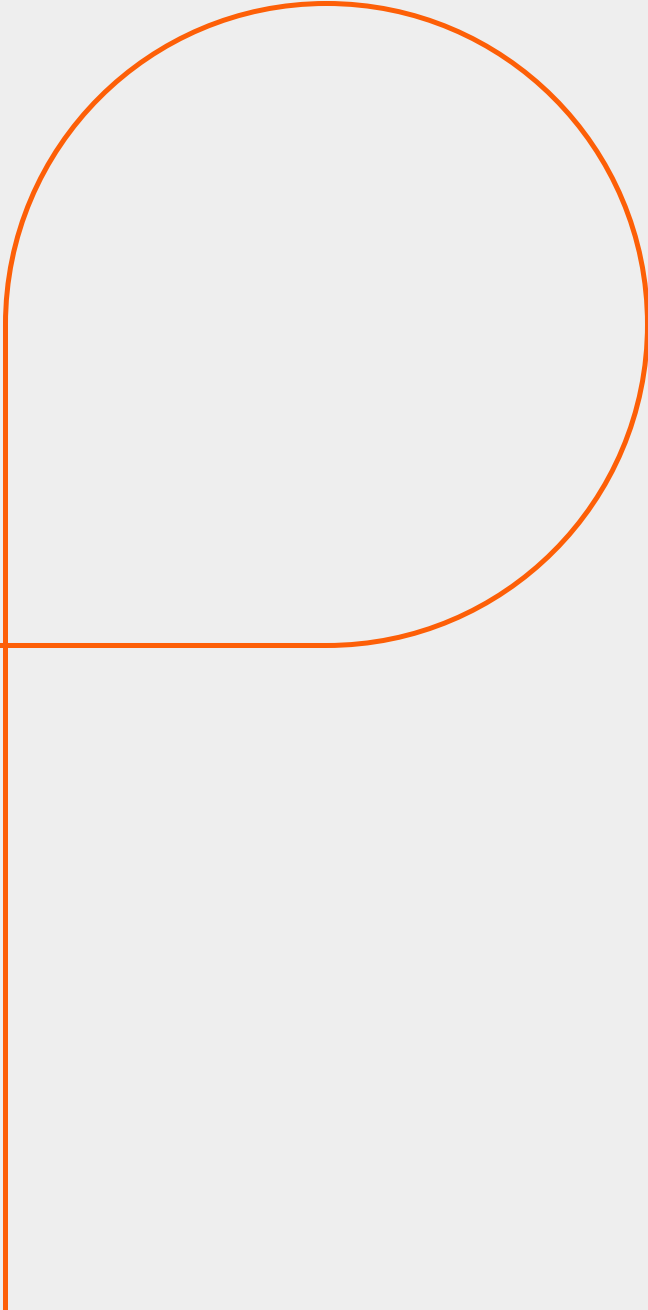
Styling the Templates

- Use the 'styles' property to define CSS classes for the template.

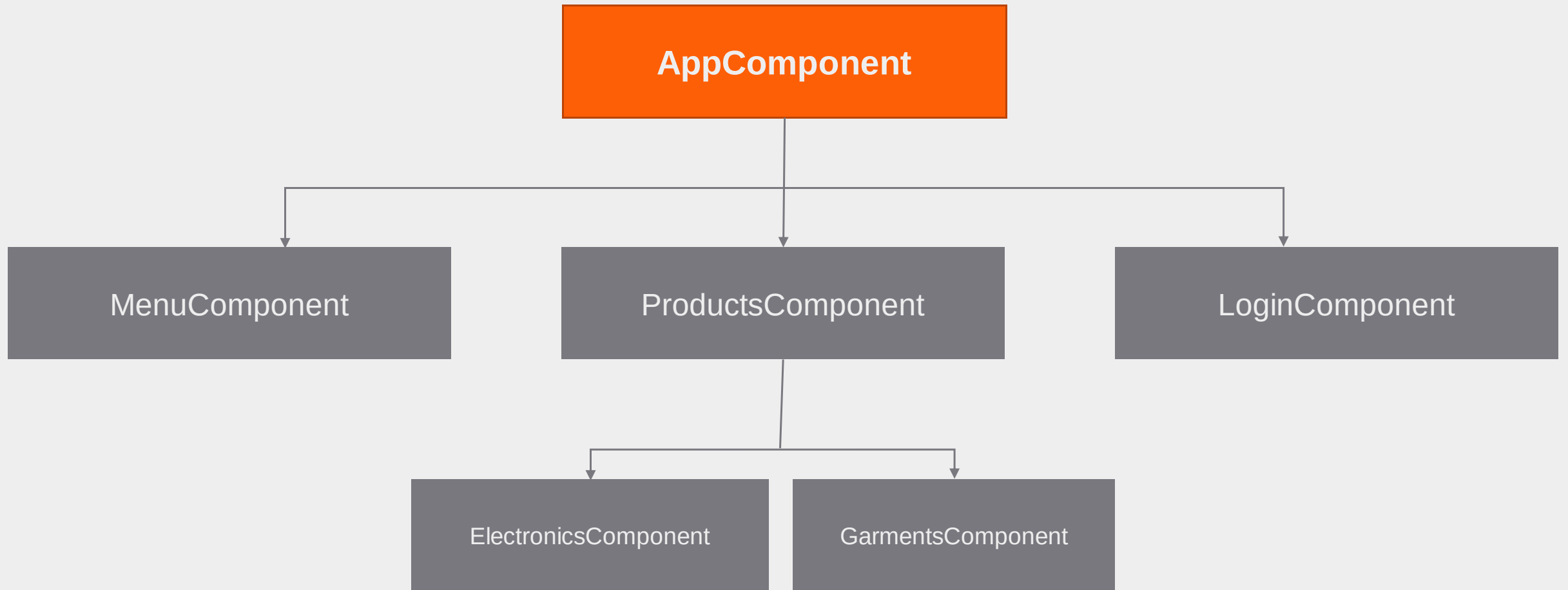
```
@Component({
  selector: 'my-app',
  styles: [`
    h1{
      color:#545454;
      background:#02A8F4;
      padding:15px;
      box-shadow:2px 2px 2px 0 rgba(0, 0, 0, 0.3);
    }
  `],
})
```

- Use 'styleUrls' to specify external CSS file.

Tree Structure of Components

A decorative graphic consisting of a horizontal orange line that extends from the left edge of the slide. At its right end, it meets a vertical orange line that extends downwards to the bottom edge. A large, thin orange circle is positioned in the upper right quadrant, with its left edge touching the horizontal line and its bottom edge touching the vertical line.

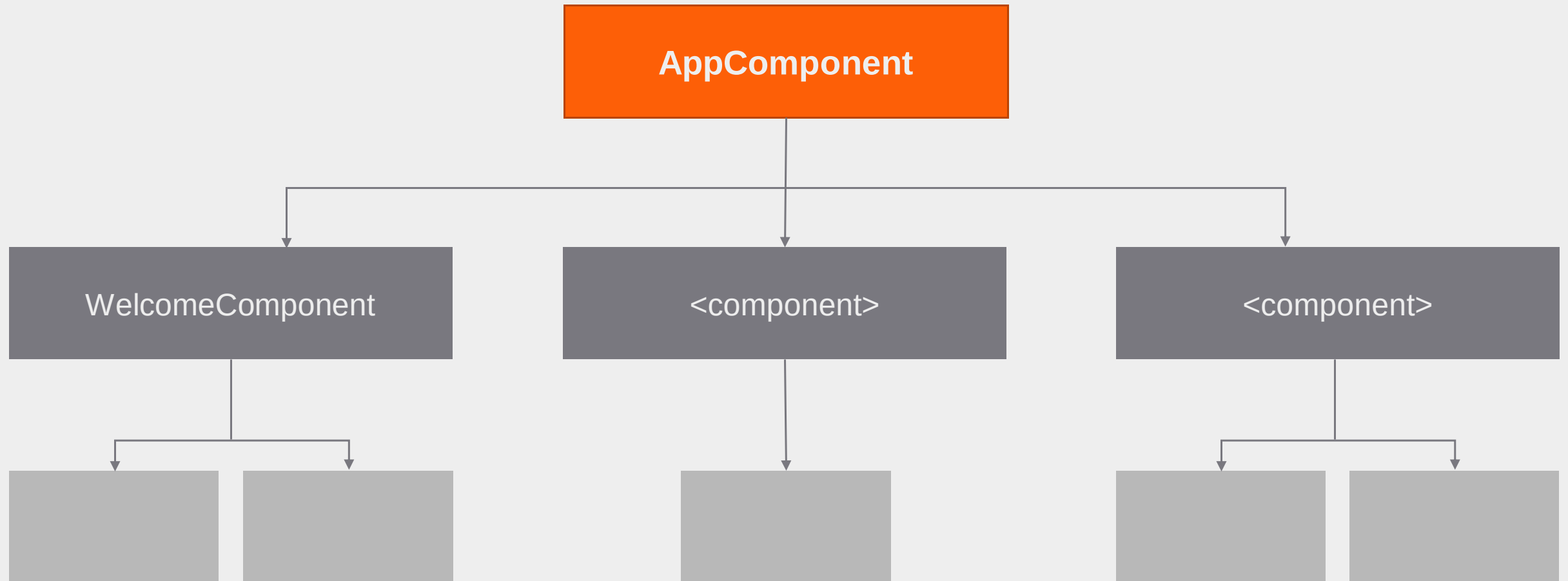
Component Tree



AppComponent

- By default the Angular project has one component AppComponent.
- This AppComponent is the Root Component by default.
- The Root Component is widely named as 'AppComponent'.
- The index page has the selector<app-root></app-root>.
- So the Angular application displays AppComponent by default.

Adding to the Component Tree



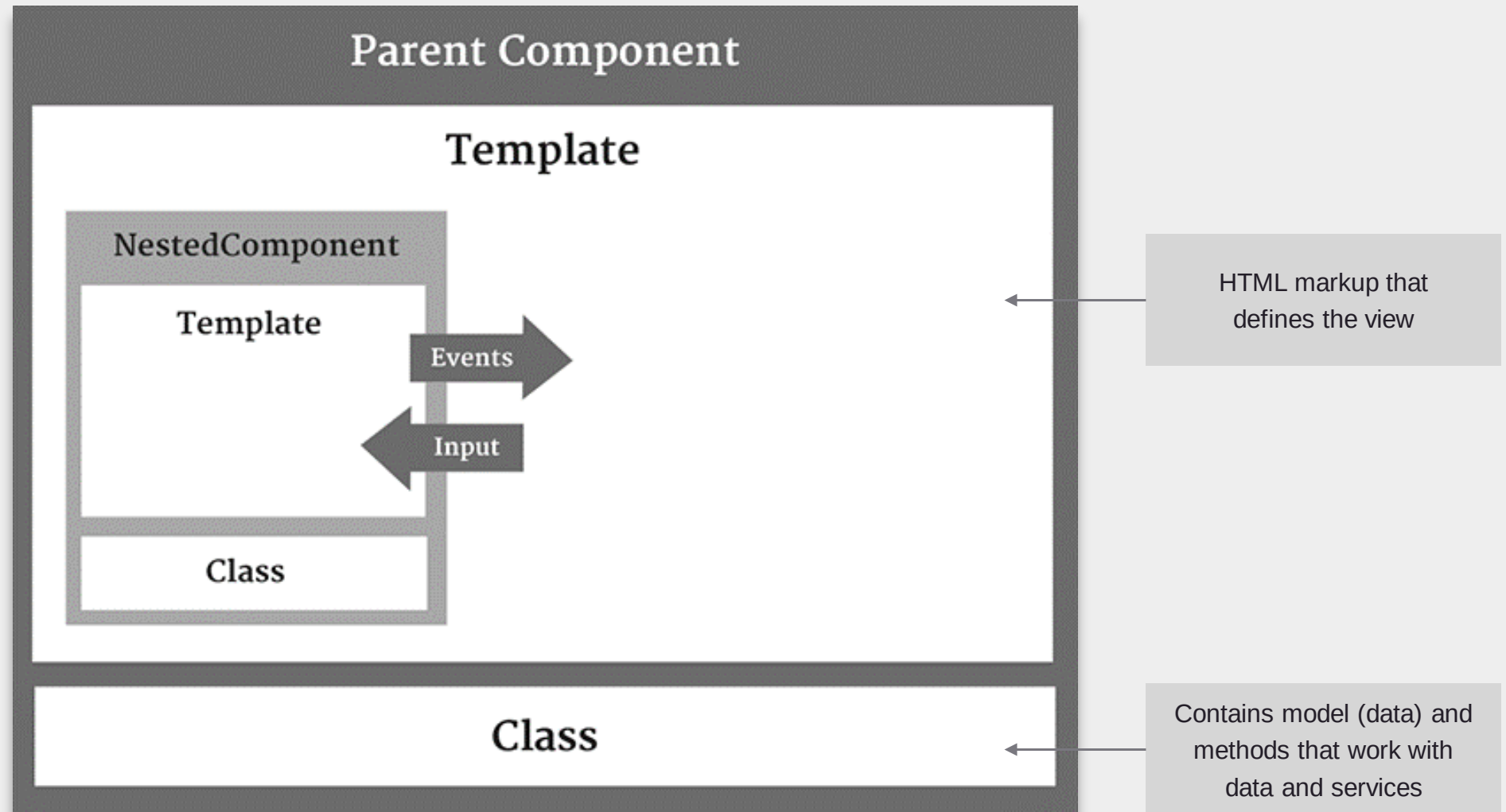
Nested Component

- Any new component that you add in app.component.html will be a Child Component of AppComponent.

```
<app-menu></app-menu>
```

```
<h1>Welcome to {{title}} project</h1>
```

```
<app-header></app-header>
```



Declare Every New Component

- Every new component must be declared in the module file corresponding to the module it belongs to.

```
@NgModule({  
  imports:      [ BrowserModule ],  
  declarations: [ AppComponent, WelcomeComponent ],  
  bootstrap:    [ AppComponent ]  
})
```

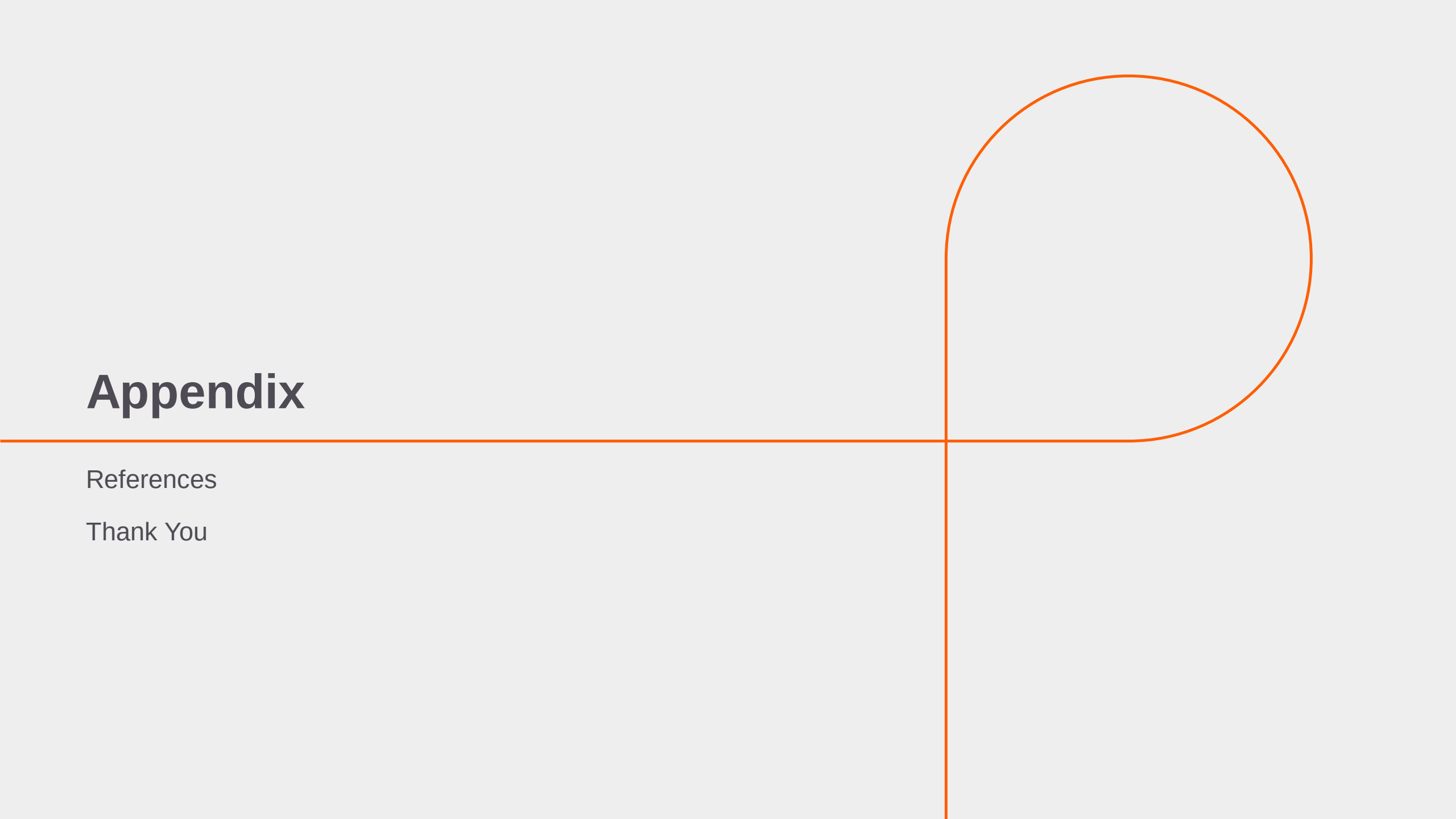


Summary: Session

With this we have come to an end of our session, where we discussed about

- How to implement a Component
- @Component properties
- How to create a Component using Angular CLI
- Tree structure of Components

Appendix



References

Thank You

References

- <https://code.tutsplus.com/tutorials/beginners-guide-to-angular-4-components--cms-29674>
- <https://www.tektutorialshub.com/angular/angular-components/#creating-the-component>
- <https://www.youtube.com/watch?v=wXmw0FxjmTc>
- <https://www.youtube.com/embed/OlokicFrD0M?feature=oembed>

Key Contacts :

Persistent University :

- Archana Ghatigar
archana_ghatigar@persistent.co.in



Thank you!

Persistent Interactive | Persistent University

